

Reviewer Pack — Minimal Hyperbolic Rank and Circuit Monotone Width Inequality...

Entry 7d0835134ae5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 12:12:04 UTC

Minimal Hyperbolic Rank and Circuit Monotone Width Inequality

Entry ID: 7d0835134ae5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 12:12:04 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry (Riemannian Metrics) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every boolean circuit C with n inputs, the minimal hyperbolic rank of its associated Riemannian metric is linearly correlated with its circuit monotone width, such that $\min_rank(M(C)) = \Theta(w_mon(C))$, where $M(C)$ is the Riemannian metric corresponding to C .

Rationale (proposer's reasoning):

Hyperbolic geometry has been shown to be a powerful tool in understanding the local structure of complex networks. If this conjecture holds, it suggests that hyperbolic metrics could provide insights into the complexity of boolean circuits, potentially leading to new circuit lower bounds or algorithmic improvements.

Taxonomy category: HyperbolicGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 144d40933b04e997

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between minimal hyperbolic rank and circuit monotone width is greater than or equal to 0.8 for at least 80% of the seeds, with a mean absolute difference between ranks and widths less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal hyperbolic rank" AND "Riemannian metric" AND boolean circuit complexity - "circuit monotone width" AND Riemannian metrics AND Hyperbolic geometry - " $\theta(w_{\text{mon}}(C))$ " AND linear correlation AND circuit monotone width in Hyperbolic Geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_circuit(n):
        if n == 1:
            return ['0']
        elif n == 2:
            return ['0', '1', 'XOR(0, 1)']
        else:
            subcircuit = generate_random_circuit(random.randint(1, n-1))
            gate = random.choice(['AND', 'OR'])
            return [f'{gate}({subcircuit[i]}, {subcircuit[i+1]})' for i in range(len(subcircuit)-1)]

    def compute_ranks(circuits):
        ranks = []
        for circuit in circuits:
            rank = 0
            for gate in circuit.split(','):
                if 'AND' in gate or 'OR' in gate:
                    rank += 1
            ranks.append(rank)
        return ranks

    def compute_widths(circuits):
        widths = []
        for circuit in circuits:
            width = len(circuit.split(',')) - 1
            widths.append(width)
        return widths
```

```

n_values = [5, 10, 15, 20, 30, 40]
ranks = []
widths = []

for n in n_values:
    circuits = generate_random_circuit(n)
    ranks.extend(compute_ranks(circuits))
    widths.extend(compute_widths(circuits))

if not ranks or not widths:
    return {
        "metric_name": "correlation",
        "metric_value": 0.0,
        "instances_tested": len(ranks),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "empty_circuit"
    }

correlation = sum((r - mean(ranks)) * (w - mean(widths)) for r, w in zip(ranks, widths)) / (len(ranks) * len(widths))
mean_diff = abs(mean(ranks) - mean(widths))

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": len(ranks),
    "n_max": max(n_values),
    "conjecture_holds": correlation >= 0.8 and mean_diff <= 3,
    "counterexample": ""
}

def mean(lst):
    return sum(lst) / len(lst)

def std(lst):
    avg = mean(lst)
    variance = sum((x - avg) ** 2 for x in lst) / len(lst)
    return math.sqrt(variance)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [random.randint(1, 1000) for _ in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_threshold\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c83e7e6.py", line 93, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7c83e7e6.py", line 67, in run_trial
    correlation = sum((r - mean(ranks)) * (w - mean(widths)) for r, w in zip(ranks, widths)) / (len(ranks) - 1)
                  ~~~~~^~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it was unable to compute the required metrics for the conjecture. | next: Re-run the test with appropriate error handling and checks to ensure that it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13671
2	propose	ollama_remote	glm4:latest	0	0	12661
3	preregistration	ollama_remote	glm4:latest	0	0	9402
4	novelty	ollama_remote	glm4:latest	0	0	9695
5	novelty	ollama_remote	glm4:latest	0	0	8874
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36826
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12463
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9074
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10104
10	judge	ollama_remote	glm4:latest	0	0	14509

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137279 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/7d0835134ae5.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7d0835134ae5.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7d0835134ae5.tar.gz (if generated)